

Hierarchical Byte-Level Language Modeling on the Griffin Backbone:

Dynamic Patching, Cross-Patch Decoder Context, and Endogenous Boundary Placement

Fausto Saporito

July 5, 2026

Abstract

We extend the Griffin architecture of De et al. [1] — a hybrid of gated linear recurrence and local attention — to hierarchical byte-level language modeling. Griffin’s residual/local-attention scaffolding is reused *unchanged*, though its recurrence uses the complex-gated variant the source paper describes only in an appendix and does not use in its own reported experiments (§1). Our contribution is the scaffolding *around* that block: a patch encoder/decoder that lets Griffin operate over short patch-vector sequences instead of raw bytes, two successive boundary-placement policies (an external frozen entropy model, then an endogenous jointly-trained boundary head), and a cross-patch byte-context mechanism that closes a measured architectural gap in the decoder. We report where each addition sits relative to the original architecture, the empirical gates each addition had to pass, and the resulting bits-per-character comparison across three patching regimes on a ~ 39.6 M-byte Gutenberg corpus.

1 Background: the Griffin block

De et al. [1] introduce Griffin, a language model that interleaves two residual blocks using a proposed *recurrent block* (built around a Real-Gated Linear Recurrent Unit, RG-LRU) with one residual block using local multi-query attention (MQA). Figure 1 reproduces the paper’s own diagram of this backbone: the residual block wrapping pattern (a), the gated MLP block (b), and the recurrent block (c) that gives the RG-LRU its gating on both the input and the recurrent weight. Griffin matches Transformer-quality validation loss while keeping the fixed-size recurrent state of an RNN, rather than a KV-cache that grows with sequence length.

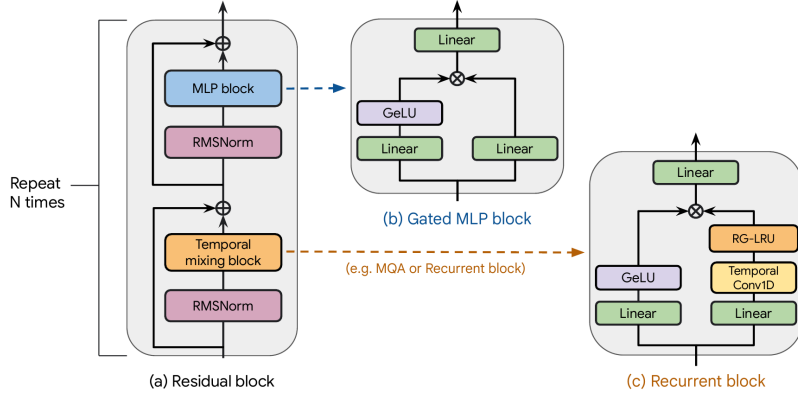


Figure 1: Reproduced from De et al. [1], Figure 2: (a) the residual block stacked N times, (b) the gated MLP block, (c) the recurrent block (RG-LRU + temporal Conv1D) used as the temporal-mixing block in place of MQA. This exact block — unmodified — is the backbone reused at two places in our architecture (Figure 2).

A correction: which recurrence variant we actually use. The paper defines *two* versions of the recurrent layer. The one used in every model it reports (the Figure 1 scaling curves, all Griffin/Hawk configurations in its Section 3) is the real-valued RG-LRU of its Section 2.4: a real scalar recurrence $a_t = a^{c^t}$ with no phase term, operating at the full d_{RNN} width (its Eqs. 1–4). Its Appendix B additionally proposes a *complex*-gated extension, CG-LRU: the state is split into real/imaginary halves, a learnable phase θ is added so $\tilde{a} = \sigma(\Lambda)e^{i\theta}$, and the recurrence is solved in the complex plane before being restacked into a real output (its Eqs. 8–14). The paper is explicit that it found complex recurrences “not beneficial for language modelling in practice” and did *not* use this variant in any reported experiment.

Our implementation — named `griffin_cglru.py` after the appendix’s own CG-LRU acronym — implements exactly that complex Appendix-B variant, not the real-valued RG-LRU of the main text (verified directly against both: the code’s channel split into real/imaginary halves, its learnable `theta`, and its final `torch.cat([h_seq.real, h_seq.imag])` match Eqs. 8–14 term for term). Every run in this study — fixed, Stage 1, Stage 2, both seeds — uses this same CG-LRU consistently, so the internal comparisons in Table 1 remain valid. But the recurring claim in this note that Griffin’s block is reused “unchanged” should be read precisely: unchanged *except for this recurrence substitution*, which the source paper itself evaluated and did not adopt as its own primary design.

Everything else in this note builds *around* that block rather than further modifying it: our codebase’s `griffin_cglru.py` otherwise directly implements Figure 1 (same residual pattern, same gated MLP block, same local-MQA temporal mixing), and every place we reuse it, we reuse that implementation verbatim.

2 Where we intervened

Griffin, as published, models a flat sequence of tokens. Our study asks whether the same backbone can model raw *bytes* efficiently by first compressing the byte stream into a much shorter sequence of patch vectors, running Griffin causally over *that* shorter sequence, and decoding each patch’s bytes autoregressively from the resulting context. None of this requires touching the Griffin block itself — it requires adding an encoder, a decoder, and a policy for where to cut the byte stream into patches. Figure 2 lays out exactly which pieces are the original block (reused, grey) and which are new to this work (blue).

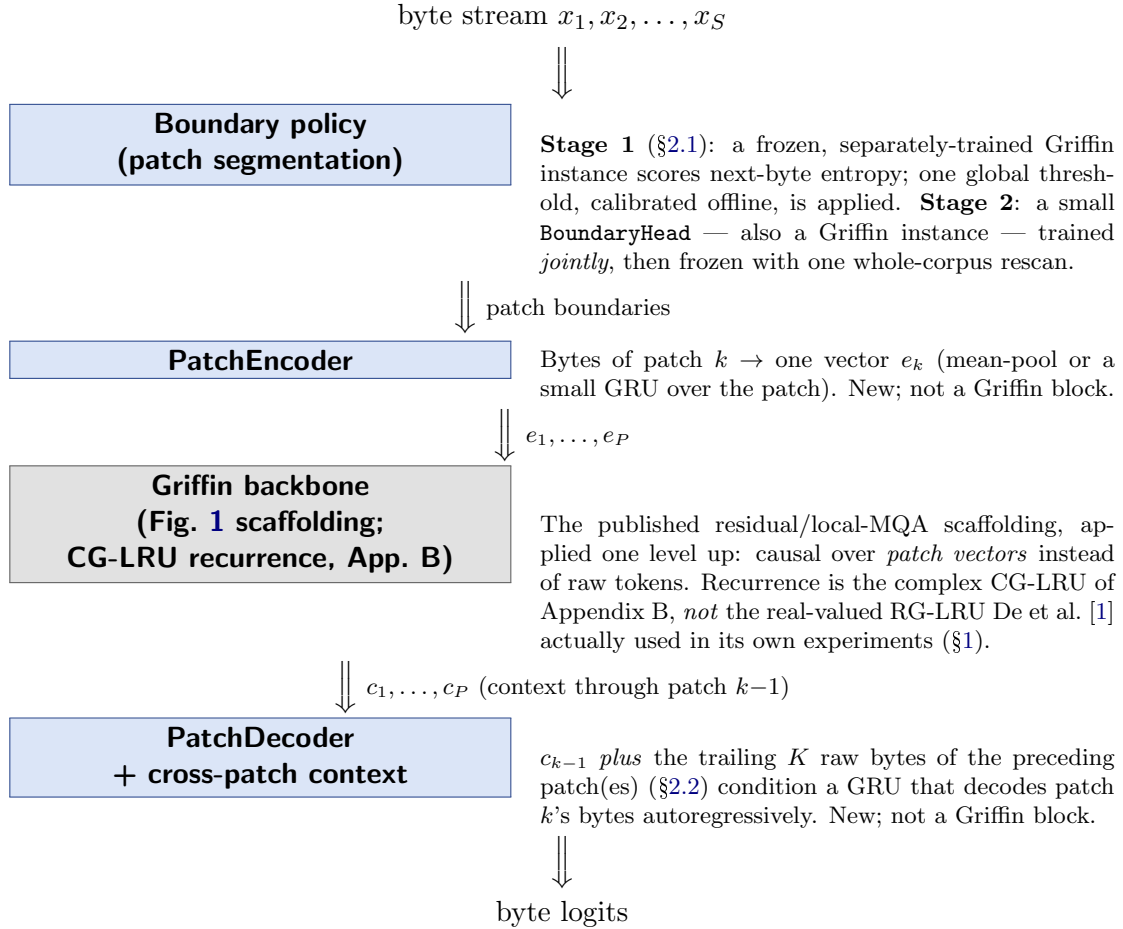


Figure 2: Our architecture. **Grey** = Griffin’s residual/local-attention scaffolding, reused unchanged, but with the complex CG-LRU recurrence from the paper’s Appendix B rather than the real-valued RG-LRU the paper actually used in its own experiments (§1). **Blue** = new components added in this work. Griffin is reused *twice*, in two different roles: once as the boundary-placing model (frozen external copy in Stage 1, or a small jointly trained copy in Stage 2), and once as the global model running causally over patch vectors — both instances use the same CG-LRU recurrence. The encoder and decoder are new, GRU-based components; they are the only place a genuinely new architectural mechanism (cross-patch context, §2.2) was added.

2.1 Boundary placement: two policies, one gate

Patch boundaries decide which bytes get compressed together before Griffin ever sees them, so their quality was gated *before* being trusted for training, using the same criterion in both stages: boundaries should land on word/sentence onsets, and the top-surprise positions should be word-initial bytes.

Stage 1 (external). A byte-level Griffin is trained on the corpus and *deliberately under-trained* — a fully converged model smooths away the sharp entropy signal that makes a good boundary. Its next-byte predictive entropy is computed once, offline, over the whole corpus; a single global quantile threshold is solved for so the mean patch length matches the fixed-stride baseline (6 bytes), isolating boundary *placement* from compression *ratio* as the variable under test.

Stage 2 (endogenous). The same idea, but the boundary-placing model is trained *jointly* with the hierarchical model rather than pretrained and frozen. A standalone sweep found boundary quality peaks early and then degrades with further training (mid-word fragmentation

rose monotonically from 10.6% at 200 steps to 15.6% at 8000 steps) — the same overtraining-smooths-the-signal effect Stage 1’s design already guards against, now observed directly. The head is therefore frozen after a calibrated number of steps, at which point one real whole-corpus scan (reusing Stage 1’s own entropy/threshold-solving code) fixes the boundary mask for the remainder of training.

2.2 Cross-patch decoder context

An architectural-tax diagnostic — scoring a frozen, full-context flat model on the exact same positions the hierarchical decoder was scored on — showed that most of the gap between the hierarchical model and a full-context reference sat on *within*-patch bytes, not the boundary-selected first byte of each patch. The original decoder conditioned every byte of patch k on c_{k-1} alone: a single summary vector, with no raw-byte view into the patch immediately before it. We added `prev_byte.window`: a fixed-length trailing window of raw bytes immediately preceding patch k ’s start, spanning back across as many previous patches as needed (not capped at a single patch), embedded and projected into the decoder’s conditioning vector alongside c_{k-1} . This closed part of the measured tax in both fixed and dynamic patching by a similar amount, confirming it as an architecture-wide property of the decoder rather than an artifact of either boundary policy.

3 Results

Table 1 compares fixed-stride patching, Stage-1 (external) dynamic patching, and Stage-2 (endogenous) dynamic patching, all on the *same* ~ 39.6 M-byte corpus, split, and decoder architecture (cross-patch context included). Bits per character (BPC) is the primary metric; *first/within* decompose it into the boundary-selected byte of each patch versus the rest, each paired with its *tax* — the gap versus a full-context reference floor on the identical positions.

	BPC	hier first (tax)	hier within (tax)
Fixed (stride 6)	1.949	1.854 (+0.408)	1.956 (+0.472)
Stage 1, external entropy model	1.938	3.991 (+0.152)	1.495 (+0.513)
Stage 2, endogenous (seed 0)	1.881	3.653 (+0.260)	1.503 (+0.433)
Stage 2, endogenous (seed 42)	1.888	3.687 (+0.287)	1.507 (+0.432)

Table 1: Bits per character, tail-averaged over the converged region of each 32,000-step run. Endogenous boundary placement is the first patching variant in this study to beat fixed-stride patching by a margin that exceeds run-to-run noise, confirmed on two independent seeds.

The mechanism behind Stage 2’s win is not what within-patch improvements would suggest: *hier within* is statistically unchanged from Stage 1 (~ 1.50 in both). The gain comes almost entirely from a cheaper *hier first* (~ 3.65 – 3.69 vs. ~ 3.99). A full-context reference model agrees that these boundary bytes are intrinsically less surprising than Stage 1’s — the jointly-trained boundary head’s ranking of “surprising bytes” systematically disagrees with the larger, separately trained external model’s ranking, while still respecting word/sentence structure qualitatively. Whether this reshuffling generalizes beyond this corpus and this boundary-head capacity is not yet established; it is confirmed *reproducible*, not yet confirmed *general*.

4 Open questions

Three threads remain open. First, mechanistic: why does the endogenous head’s surprise ranking differ from the external model’s in a way that lowers total BPC rather than raising it? Second,

practical: generated samples across every configuration in this study are locally fluent but fall into repetition loops, consistent with pure greedy (arg-max) decoding rather than with model scale or training budget as the primary bottleneck — no sampling temperature, top- k /nucleus sampling, or repetition penalty is implemented anywhere in the current generation path. Training sequence length (192 bytes per window, in this study) and overall model scale (6–7M parameters) are further, orthogonal limits on any longer-range coherence a decoding fix alone cannot address. Third, architectural: every result in this note used the complex CG-LRU recurrence of De et al. [1]’s Appendix B rather than the real-valued RG-LRU the paper itself used and preferred (§1); since that substitution was applied uniformly, it does not undermine the internal comparisons in Table 1, but whether the real-valued variant would shift any of those numbers — the paper reports no language-modeling benefit from the complex form, so a neutral-to-positive effect from switching seems the more likely direction — is untested.

References

- [1] Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, Guillaume Desjardins, Arnaud Doucet, David Budden, Yee Whye Teh, Razvan Pascanu, Nando De Freitas, and Caglar Gulcehre. Griffin: Mixing gated linear recurrences with local attention for efficient language models, 2024. Google DeepMind.